Auto formalisation of Chaitin and of the surprise incompleteness Theorem

Thierry Coquand

Computer Science and Engineering Department, University of Gothenburg, Sweden,
coquand@chalmers.se

June 7, 2026

Abstract

This is a continuation of a previous report on an experiment in autoformalisation of Gödel's second incompleteness theorem in Agda using Claude. Using the framework built in this experiment, Claude could "auto-formalise" Chaitin's proof of the first incompleteness theorem and then the Kritchman-Raz surprise examination paradox version of the second incompleteness.

As the first experiment, the project provides a case study of the strengths and limitations of current large language models in mathematics. Since Chaitin's proof involves coding programs, Claude had to represent code as ternary string and could build autonomously a parser and a continuation stack evaluation machine. The fact that we can simulate computations as expected is not completely trivial and we suggested a Gandy/Howard majorisation argument, that Claude had no problem to follow.

The resulting formalisation clarifies a number of details left implicit in the original presentation and provides a fully machine-checked proof of these arguments for Church's Basic Recursive Arithmetic.

Introduction

This is a continuation of a previous report [4] on an experiment in autoformalisation of Gödel's second incompleteness theorem in Agda using Claude. Using the framework built in this experiment, Claude could "auto-formalise" Chaitin's proof of the first incompleteness theorem [2, 3] and then the Kritchman-Raz surprise examination paradox version of the second incompleteness [9].

We did not write a single line of Agda and it was using a "spartan" version, without any tactics, and library. We just want first to point out some subtle point about the arguments, that have to be formulated in Skolem arithmetic and on some metatheoretic remark about the significance of the proof for constructive mathematics. We then end with a remark on the form of the "surprise" argument, which as we formulated is actually closer to the sorites/heap of sand paradox than the surprise examination paradox. We let then Claude document more in details the formalisation.

1 How to encode computation in Basic Recursive Arithmetic

Chaitin's proof needs an internal representation of computation. This is subtle since we *cannot* define internally an evaluation function: it will look like Ackermann function and is not primitive recursive arithmetic. Instead Claude wrote *autonomously* and without any suggestion from my

part a continuation stack small step evaluation machine. The problem is then to show that for a given function f, is we run the machine on $code(f)$ on an argument x, we can find n big enough such that, after n steps, the machine halts with the value $f(x)$.

This is counter-intuitive since there is on the other hand no eval function. Claude was stuck in producing n as a function of f. We suggested to use the Gandy/Howard method, which produces *internally* the number of evaluation steps for computation. With *only* this suggestion, Claude was able to produce n as an internal function, and then to finish the proof by induction on how f is built.

2 Potential interest for constructive mathematics

In constructive mathematics, the primary notion is the notion of *total* function: a function is "by nature" total. It is not so easy to find actual examples where the notion of *partial* function can play a role. In Chaitin's proof, it plays a crucial role (and this in the constructive framework of Binary Recursive Arithmetic). At some point in the argument, we have to exhibit a program which has to be *short* w.r.t. a certain threshold, and for producing this short program, it is essential to use a looping program, which may be a priori partial: we enumerate all possible derivable formuale, $th(0)$, $th(1)$, ... until we find one of a required form. This program is ensured to terminate when one uses it, but, in this is the key, the length of the computation is not taken into account in its size. (This is a size manifestation of Blum's speed-up phenomenon [1]: allowing a partial recursive program that is kept total only by an *external* proof of termination — rather than one that must internally certify its own halting — can shorten the program unboundedly, since program size and running time are independent complexity measures.)

3 The statement of Chaitin's result

In the setting of Skolem arithmetic, Chaitin's result gets a computational interpretation. It states the existence of a function g such that

$$th(x) = code(K(r) > L*) \Rightarrow th(g(x)) = code(0 = 1)$$

for a certain threshold $L*$ which is a concrete, computable, natural number. Here, as usual $K(r) > L*$ means that r cannot be named by a code p, such that $log_3(p) \leqslant L*$. The theorem is not merely that $K(r) > L*$ is unprovable. It produces an explicit proof-transformer.

This is somewhat counter-intuitive since there should clearly be numbers that cannot have a short description. This is precisely this which is used by Kritchman and Raz [9] to provide a proof of the second incompleteness theorem.

4 The proof of the second incompleteness

Our argument for the second incompleteness follows [9], but it does not proceed by counting. If N is big enough, we know, by a pigeon-hole priciple, that we cannot have all numbers in $[0, N]$ having short description. This is $S(0)$. It is then not possible that all numbers in $[1, N]$ have short description. The intuitive reason is that, otherwise, we could prove that 0 has no short description, and then we can use Chaitin's Theorem to produce a contradiction. (The exact argument is detailed below.) This is $S(1)$. We can then prove in this way by external induction on r, the statement $S(r)$ that it cannot be the case that all numbers in $[r, N]$ have a short description. We then obtain $S(N + 1)$, which is a contradiction.

We think this argument can also be seen as a variation of the sorites/heap of sand: $[0, N]$ is the heap of sand. If we take away 0, it stays a heap of sand, this is $S(1)$. And if we proceed in this way, we get $S(2), S(3), \ldots$ until we have taken away all grains. What is striking is that N, like $L*$ is a concrete, computable number.

5 Conclusion

We think that present coding LLM, such as Claude, are now perfect tools to help to understand all details of a given mathematical paper. As they are now, they require constant supervisions, which is fine for small examples such as the ones we have analysed. Finally, as the previous experiment, the language of type theory was perfect for expressing this kind of concrete mathematics, manipulating evaluation machine and coding. The proofs are then directly represented, without any need of tactics. We do not need at this level function extensionality and quotient types. One key research question may be the design formal of systems where the same can be done for parts of mathematics dealing with non-combinatorial object.

Authorship note. Consistent with the autoformalisation theme of this project, the rest of this text, Sections 5–11, were generated by Claude. (I did not write a single line there.) The mathematical content, references, and final text were reviewed by the author.

6 The statement

We work entirely inside the system T of Basic Recursive Arithmetic in the formulation of Guard, with Shoenfield's notation; the axioms, the numerals $\mathbf{k}_n = \mathbf{s}^n(\mathbf{O})$, the encoding $\mathit{code}(\cdot)$, the verifier $\mathbf{th} \in \mathit{Fun}_1$ (with $\mathit{Thm}(\mathbf{th}(\mathit{code}(d)) = \mathit{code}(A))$ whenever d codes a derivation of A), the numeral functor $\mathbf{num} \in \mathit{Fun}_1$ (with $\mathit{Thm}(\mathbf{num}(\mathbf{k}_n) = \mathit{code}(\mathbf{k}_n))$) and the derivability-internalising Theorems 12 and 13 are all as set up in the companion paper [4], to which we refer for every detail of the system. We write $\mathit{Thm}(A)$ for "A is a theorem of T".

As in [4], the consistency of T is the *open* formula

$$\mathrm{Con}_T \;\equiv\; \neg\,(\mathbf{th}(\mathbf{x}_0) = \mathit{code}(\mathbf{O} = \mathbf{s}\,\mathbf{O})),$$

with the single free variable $\mathbf{x}_0$. Since T has no object quantifiers, a theorem is an open formula read as implicitly universally quantified over its free variables, so Con_T says: *for every term* $\mathbf{x}_0$*,* $\mathbf{x}_0$ *does not code a derivation of* $\mathbf{O} = \mathbf{s}\,\mathbf{O}$. The theorem we prove is the same meta-implication that the Guard-route proof establishes,

$$\boxed{\mathit{Thm}(\mathrm{Con}_T) \implies \mathit{Thm}(\mathbf{O} = \mathbf{s}\,\mathbf{O}),}$$

i.e. if T proves its own consistency then T is inconsistent. The arrow $\implies$ is a meta-level implication between derivability judgements, realised in Agda as a total function transforming any proof of Con_T into a proof of $\mathbf{O} = \mathbf{s}\,\mathbf{O}$. The consistency hypothesis is consumed exactly once per step of the descent of §8, and is the *only* assumption: every other ingredient is constructed.

7 Chaitin's first incompleteness theorem, internalised

Chaitin's proof of the first incompleteness theorem [2, 3] replaces the liar sentence by Berry's paradox and Kolmogorov complexity. We need this argument not as a metatheorem but as a single T implication, with the underlying universal machine realised as an honest object-level program. This section describes that realisation in ordinary computational terms; the detailed Agda construction is part of the development cited in §9.

7.1 Programs as ternary numbers

A *program* is simply a natural number. Read a number p in base 3; its digits, drawn from the three-symbol alphabet $\{1, 2, 3\}$, form a string, and that string is fed to a fixed parser **parse** that turns it into a syntax tree — an expression of the object language. The *length* $|p|$ of the program is the number of base-3 digits, so

$$|p| \le n \quad\Longleftrightarrow\quad p < 3^{\,n+1}.$$

Thus the enumeration of programs is the *identity*: the k-th program is the number k, and the finite set of programs of length at most $L^\star$ is exactly the initial segment

$$\{\, p : p < N \,\}, \qquad N := 3^{\,L^\star+1}.$$

This is the single simplification that makes the whole argument feasible: there is no separate enumeration to verify — "the set of short programs" *is* an interval of numbers, and quantification over it is bounded quantification over $\{p < N\}$.

7.2 The universal interpreter as a CK machine

The interpreter **U** that runs a parsed program is realised as a small abstract machine of the Landin CK kind. A *configuration* is a pair

$$\langle\, c,\ \kappa \,\rangle,$$

where c is the expression currently under evaluation (the *control*) and κ is a *continuation stack* recording the contexts still to be returned to. A one-step transition function **step** fires a single reduction rule: it either decomposes c, pushing the deferred work onto κ, or, when c has reached a value, pops the top frame of κ and plugs the value into it. Running the program for l steps is the l-fold iteration $\mathbf{step}^{\,l}$ started from the initial configuration; the machine halts when the control is a value and the stack is empty, and the output is read off that final configuration. Around this evaluator we wrap one more layer, a bounded μ-search loop, so that the outer behaviour of **U** matches the informal recipe "enumerate $\mathbf{th}(0), \mathbf{th}(1), \ldots$ until a fitting proof is found" used by the diagonal program below.

For a program p we write

$$\mathbf{def}_p(l, u) \ \equiv\ \text{"running } p \text{ for } l \text{ steps halts with output } u+1\text{"}$$

(the $+1$ separates "halted with value u" from "not yet halted", coded by 0); $\mathbf{def}_p \in \mathit{Fun}_2$ for each p. Folding the bounded disjunction of the $\mathbf{def}_p$ over the finite set $\{p < N\}$ gives a single object function $\mathbf{CK} \in \mathit{Fun}_2$ with

$$\mathbf{CK}(u, x) = \mathbf{O} \iff \text{some program } p < N, \text{ run for } x \text{ steps, outputs } u+1,$$

i.e. $\mathbf{CK}(u, x) = \mathbf{O}$ expresses $K(u) \le L^\star$ at run-length x, and the Kolmogorov bound

$$K(u) > L^\star \quad \text{is expressed by} \quad \mathbf{CK}(u, x) \neq \mathbf{O}$$

with x a free run-length variable.

7.3 Term-valued fuel: a Gandy–Howard majorant

The delicate point is not the diagonal argument — which is a faithful rendering of Chaitin's — but the fact that the interpreter must be run *inside* a **th**-fact. Since **th**, **U** and **step** all take object terms, the number of steps appearing in such a fact must itself be an object *term*, not a meta-level natural; and Theorem 13 of [4] internalises only statements about T terms. A completeness statement of the naive form "for every numeral $\bar{n}$ large enough, $\mathbf{step}^{\,\bar{n}}$ on the program halts" is therefore useless: it is quantified over meta-naturals and cannot be fed into an internal derivation. What is needed instead is a completeness statement with a closed *combinator* in the fuel slot,

$$(x : \mathit{Term}) \to\ \mathit{Thm}\left(\mathbf{step}^{\,\mathbf{fuel}(f)(x)}(\mathrm{init}(f, x)) = \mathrm{halt}(f \cdot x)\right),$$

where $\mathbf{fuel}(f) \in \mathit{Fun}_1$ is built once and for all from f.

Proving such a statement by induction on the combinator tree of f stalls at the primitive-recursion node, where one would need numerical control over the running time of the recursive calls — information not available at the derivation level. The resolution adapts the classical *majorisation* method of Howard and Gandy [7, 5]: one defines, by primitive recursion mirroring the structure of f, a closed combinator $\mathbf{fuel}(f)$ that provably *majorises* the actual number of steps any run of f consumes at every term input. The induction then carries the stable shape "the run completes at *any* fuel $\ge \mathbf{fuel}(f)(x)$", which descends through the recursion node because **fuel** is itself recursive in step with f. The μ-search loop needs the same idea one level higher, with a majorant $\mathbf{fuel}_\mu$ stacked on top of **fuel** by a fuel-composition lemma. The closed term that finally occupies the fuel slot is $\mathbf{fuel}_\mu(\cdots) + \mathbf{fuel}(\cdots)$. Without this majorisation there would simply be no term to put there.

7.4 The diagonal program and the internal implication

Chaitin's short program $g_{L^\star}$ enumerates $\mathbf{th}(0), \mathbf{th}(1), \ldots$ until it meets the code of a statement of the form $K(?) > L^\star$, and then outputs the witnessed number ?. Its size is a constant plus the number of digits needed to write $L^\star$, that is $c + \log L^\star$, which is below $L^\star$ once $L^\star$ is large. The program $g_{L^\star}$ is obtained as a genuine one-variable object combinator $G \in \mathit{Fun}_1$ by bracket abstraction — an honest T program, not a meta-level term transformer — so that $G \cdot w$ is literally the constructed code, as a *proved* T-equation. Let $\mathbf{out}(w)$ be the object function that reads the witness off w. The internalised Chaitin theorem is the single T implication

$$\mathit{Thm}\Big(\big(\mathbf{th}(w) = \mathit{code}(\, K(\mathbf{out}(w)) > L^\star \,)\big) \supset \big(\mathbf{th}(G \cdot w) = \mathit{code}(\mathbf{O} = \mathbf{s}\,\mathbf{O})\big) \Big), \qquad \text{(C)}$$

valid for *every* term w, with no consistency premise. In words: T proves that *if* $\mathbf{th}(w)$ is the incompressibility statement about w's own read-back, *then* the code $G \cdot w$ is a T-proof of $0 = 1$. The deduction lives entirely inside the object logic as an implication; the run of the interpreter on $g_{L^\star}$, with the Term-fuel of §7.3, is what closes the positive leg of (C). This is the only place where the evaluator machinery is needed; the descent of the next section uses (C) purely as a black box.

Remark 7.1 (Comparison with Kikuchi's form of Chaitin's theorem). The form of Chaitin's incompleteness theorem used by Kritchman and Raz [9] is Kikuchi's [8], namely the meta-implication

$$\mathrm{Con}_T \implies \forall x \neg \mathrm{Pr}_T(\mathit{code}(\, K(x) > L^\star \,)),$$

"if T is consistent then T does not prove $K(x) > L^\star$ for any x", where Pr_T is the Σ_1 provability predicate and K is the Kolmogorov-complexity function symbol. Our implication (C) is sharper in three respects. *First*, it carries no consistency premise: it isolates the pure diagonal mechanism as an object implication, and the single appeal to Con_T is deferred to Step 6 of the descent. *Second*, it is positive and constructive rather than a negated provability statement: instead of asserting that $K(x) > L^\star$ is unprovable, it exhibits an explicit object-level proof-transformer $G \in \mathit{Fun}_1$ turning any code of a proof of $K(\mathbf{out}(w)) > L^\star$ into a code of a proof of $\mathbf{O} = \mathbf{s}\,\mathbf{O}$. *Third*, it is stated with the honest verifier $\mathbf{th}$ and the explicit bounded universal machine of §7, with $K(x) > L^\star$ the concrete open Π_1 formula $\mathbf{CK}(x, \cdot) \neq \mathbf{O}$ over $\{p < N\}$, $N = 3^{L^\star+1}$ — in particular it uses neither object quantifiers, nor the abstract Pr_T, nor Σ_1-completeness, none of which is available in T. Kikuchi's economy — for instance, that the only "monotonicity" his argument needs is the trivial $z \leq y \wedge y \leq K(x) \to z \leq K(x)$ — stems precisely from working with Σ_1 formulas and Pr_T, which bury the program execution, and its run-length witness, inside the Σ_1 definition of K; lacking object quantifiers, T must instead expose the run-length as a free fuel variable and pay for it with the genuine run-monotonicity of Step 1 below.

8 The sorites descent

Chaitin's theorem says that, for a large enough $L^\star$, no statement $K(x) > L^\star$ is provable in a consistent T, even though for each x exactly one of $K(x) \leq L^\star$, $K(x) > L^\star$ holds and the former, when true, is provable (by exhibiting and running the short program). Kritchman and Raz [9] observed that this tension can be wound into the second incompleteness theorem by an argument modelled on the surprise-examination paradox.

8.1 Induction on the stages, not counting; a sorites

Kritchman and Raz [9] argue by *counting*: writing $m = \#\{\, u \in [0, N] : K(u) > L^\star \,\}$, they prove $\mathit{Thm}(m \geq i)$ for $i = 1, 2, \ldots$ inside T, and since $m \leq N + 1$ this induction must eventually clash. We do not count. Instead, fixing $L^\star$ as in §7 and $N = 3^{L^\star+1}$, we prove directly, by external

induction on r, the open stage statement $S(r)$ of §8.2 — "not all of the days $[r, N]$ can be simultaneously described by short programs" — with base case $S(0)$ the pigeonhole below and inductive step $S(r) \to S(r+1)$ the six-step block, so that the chain $S(0) \to S(1) \to \cdots \to S(N)$ closes with $Thm(\mathbf{O} = \mathbf{s}\,\mathbf{O})$.

This has the shape of the *sorites*, or heap-of-sand, paradox, rather than a vicious self-referential circle. Picture the interval $[0, N]$ as a heap of sand: each step of the induction removes one grain and is, taken on its own, an entirely sound T derivation — one internal application of the Chaitin implication (C) reflected through a single use of consistency — so no individual grain-removal is the culprit, yet after $N + 1$ removals the heap is gone and T has proved $\mathbf{O} = \mathbf{s}\,\mathbf{O}$. The contradiction lives only in the iteration, exactly as in the heap paradox, and it is the second incompleteness theorem — the impossibility of T proving its own consistency — that explains why the seemingly innocuous step cannot in fact be taken all the way (cf. [9]: the missing premise is the consistency of T itself).

8.2 The formal realisation: six steps

In the formalisation the count m is carried by an open formula $S(r)$, indexed by a stage $r \in [0, N]$ and built as a negated big conjunction over "days". For programs $p_r, \ldots, p_N$ (each of length $\leq L^\star$, ranging over $\{p < N\}$) write

$$K(\mathbf{x}_0; p_r, \ldots, p_N) \;\equiv\; \bigwedge_{d=r}^{N} \mathbf{def}_{p_d}(\mathbf{x}_0, d),$$

"each program p_d describes the day d, at the common run-length $\mathbf{x}_0$". The stage formula is

$$S(r) \;\equiv\; Thm(\neg\, K(\mathbf{x}_0; p_r, \ldots, p_N)) \qquad \text{(for every choice of } p_r).$$

The descent climbs $S(0) \to S(1) \to \cdots \to S(N)$, one grain at a time, and the inductive step $S(r) \to S(r+1)$ is the six-step block that realises Kritchman–Raz's induction step; it follows the high-level note `clos` of the development and reuses the implication (C) as a black box.

Step 1 (run-monotonicity and the finite set). From $S(r)$ one derives, at an *independent* run-length $\mathbf{x}_1$, the implication

$$K(\mathbf{x}_0; p_{r+1}, \ldots, p_N) \supset \neg\mathbf{def}_{p_r}(\mathbf{x}_1, r),$$

the two fuels $\mathbf{x}_0, \mathbf{x}_1$ being aligned to a common bound by monotonicity of the interpreter in its step count (running longer never unsays a halt). Folding the bounded disjunction over the finite set $\{p < N\}$ rewrites the consequent as $K(r) > L^\star$, giving

$$Thm\big(\, K(\mathbf{x}_0; p_{r+1}, \ldots, p_N) \supset (K(r) > L^\star)\,\big).$$

Step 2 (encode and substitute the numeral). By Theorem 13 of [4] the Step-1 implication is internalised: one obtains a term f with

$$Thm(\mathbf{th}(f \cdot \mathbf{x}_0) = code(\, K(\mathbf{num}\,\mathbf{x}_0; p_{r+1}, \ldots, p_N) \supset K(r) > L^\star\,)),$$

the subject slot $\mathbf{x}_0$ being replaced inside the code by its numeral $\mathbf{num}\,\mathbf{x}_0$ (the consequent has no free $\mathbf{x}_0$ and is left inert).

Step 3 (internal modus ponens). An encoded, Carneiro-lifted modus ponens [4] chains Step 2 with a **th**-fact for the antecedent.

Step 4 (Theorem 13 for the conjunction). Writing the big conjunction as $K_r(\mathbf{x}_0) = \mathbf{O}$, the Σ_1-completeness direction

$$\big(K_r(\mathbf{x}_0) = \mathbf{O}\big) \supset \mathbf{th}(D\,\mathbf{x}_0) = \\ \mathit{code}(\,K(\mathbf{num}\,\mathbf{x}_0; p_{r+1}, \ldots, p_N)\,)$$

supplies that antecedent fact (Kritchman–Raz's equation (2)).

Step 5 (Chaitin). The internal Chaitin implication (C) is applied:

$$\mathit{Thm}\big(\,(\mathbf{th}(w) = \mathit{code}(K(r) > L^\star)) \supset (\mathbf{th}(G \cdot w) = \mathit{code}(\mathbf{O} = \mathbf{s}\,\mathbf{O}))\,\big).$$

Composing Steps 3–5 yields

$$\mathit{Thm}\big(\,K_r(\mathbf{x}_0) = \mathbf{O} \supset (\mathbf{th}(h \cdot \mathbf{x}_0) = \mathit{code}(\mathbf{O} = \mathbf{s}\,\mathbf{O}))\,\big).$$

Step 6 (consistency). Finally Con_T is instantiated at $h \cdot \mathbf{x}_0$ and, by contraposition, refutes the consequent — so the antecedent fails: $\mathit{Thm}(\neg\, K(\mathbf{x}_0; p_{r+1}, \ldots, p_N))$, which is $S(r+1)$. This is the single use of consistency per grain (Kritchman–Raz's equation (1)).

Base case. $S(0)$ is hypothesis-free and combinatorial. There are $N+1$ days but only N program slots, so any assignment of a program to each day forces a collision $p_i = p_j$ with $i \neq j$; the two day-conjuncts then assert that one program, at the same run-length, outputs both $\mathbf{s}^i$ and $\mathbf{s}^j$, which is impossible. Hence $\neg K(\mathbf{x}_0; p_0, \ldots, p_N)$.

Final clash. At the top stage $r = N$ the remaining conjunction is empty, hence provably true, so the Step-1 implication fires *unconditionally*: T proves $K(N) > L^\star$, the Chaitin diagonal makes its provability into a proof of $0 = 1$, and Con_T refutes it. The heap is empty and

$$\mathit{Thm}(\mathbf{O} = \mathbf{s}\,\mathbf{O}).$$

By external induction the chain $S(0) \to \cdots \to S(N)$ closes the proof. This induction is not informal: it is carried out in the metatheory by Agda, as a recursion on the meta-natural r that turns the six-step block into a function $\mathit{Thm}(S(r)) \to \mathit{Thm}(S(r+1))$ and iterates it from the base derivation of $S(0)$ up to $S(N)$.

9 The formalisation

The proof is checked in Agda, in the same `T4` development as the Guard-route proof of [4], under the global options `--safe --without-K --exact-split`, with no postulates and no holes. The headline is a single function from a derivation of Con_T to a derivation of $\mathbf{O} = \mathbf{s}\,\mathbf{O}$; Con_T is the only hypothesis, and it is the same open consistency formula as in [4].

The one genuinely non-classical piece is the Gandy–Howard majorant of §7.3. The diagonalisation is Chaitin's, but the closed object term occupying the fuel slot of the internal interpreter run has no counterpart in the informal argument, and its construction (by recursion mirroring the program, with a substituted-motive induction through the recursion node) is what lets evaluator completeness be *stated and proved* inside T. It is worth noting, finally, that Guard's formulation of Basic Recursive Arithmetic — syntax-first, with the verifier **th** and the numeral functor **num** as honest object functions — proved well suited to expressing all of these results, including the internal Chaitin implication and the surprise descent.

The development, including both proofs of the second incompleteness theorem, is available at `https://github.com/coquand/agda-godel-tree`.

References

[1] M. Blum, A Machine-Independent Theory of the Complexity of Recursive Functions. Journal of the ACM 14(2), 1967, pp. 322–336.

[2] G. J. Chaitin. Computational complexity and Gödel's incompleteness theorem. *ACM SIGACT News*, 9:11–12, 1971.

[3] G. J. Chaitin. Information-theoretic limitations of formal systems. *Journal of the ACM*, 21(3):403–424, 1974.

[4] T. Coquand. Autoformalisation of Gödel's second incompleteness theorem in Basic Recursive Arithmetic. arXiv:2606.01898, 2026. `https://arxiv.org/abs/2606.01898`.

[5] R. O. Gandy. Proofs of strong normalization. In J. P. Seldin and J. R. Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 457–477. Academic Press, 1980.

[6] J. R. Guard. Lecture notes on recursive arithmetic. Unpublished lecture notes, 1963.

[7] W. A. Howard. Hereditarily majorizable functionals of finite type. In A. S. Troelstra, editor, *Metamathematical Investigations of Intuitionistic Arithmetic and Analysis*, volume 344 of *Lecture Notes in Mathematics*, pages 454–461. Springer, 1973.

[8] M. Kikuchi. Kolmogorov complexity and the second incompleteness theorem. *Archive for Mathematical Logic*, 36:437–443, 1997.

[9] S. Kritchman and R. Raz. The surprise examination paradox and the second incompleteness theorem. *Notices of the American Mathematical Society*, 57(11):1454–1458, 2010.

[10] J. R. Shoenfield. *Mathematical Logic*. Addison–Wesley, Reading, MA, 1967.